

C++学习笔记6

2022年3月19日 20:54

- 顺序容器

- 顺序容器概述

- 所有顺序容器都提供了快速访问元素的能力，但是这些容器都在以下方面有不同的性能折中

表 9.1: 顺序容器类型	
vector	可变大小数组。支持快速随机访问。在尾部之外的位置插入或删除元素可能很慢
deque	双端队列。支持快速随机访问。在头尾位置插入/删除速度很快
list	双向链表。只支持双向顺序访问。在 list 中任何位置进行插入/删除操作速度都很快
forward_list	单向链表。只支持单向顺序访问。在链表任何位置进行插入/删除操作速度都很快
array	固定大小数组。支持快速随机访问。不能添加或删除元素
string	与 vector 相似的容器，但专门用于保存字符。随机访问快。在尾部插入/删除速度快

- 向容器添加或从容器删除的代价

- 非顺序访问容器元素的代价

- 除了固定大小的array外，其他容器都提供高效、灵活的内存管理，我们可以添加和删除元素，扩张或者收缩容器的大小

- 顺序容器特征

- string和vector将元素保存在连续的内存空间中
 - list和forward——list相比于vector deque array，这两个容器的内存开销也很大
 - deque是一个更为复杂的数据结构，与string和vector类似
 - deque支持快速的随机访问。
 - 与string和vector一样，在deque的中间位置添加或删除元素的代价可能更高
 - 在deque的两端添加或删除元素都是很快的，与list或forward——list添加删除元素的速度相当
 - 与内置数组相比，array是一种更安全、更容易使用的数组类型，与内置数组类似，array对象大小是固定的，因此array不支持添加和删除元素以及改变容器大小的操作

- 确定使用哪种顺序容器

- 通常，使用vector是最好 的选择，除非你有很好的理由选择其他容器
 - 如果你的程序有很多小的元素，且空间额外开销很重要，不要使用list或forward——list。
 - 如果程序要求随机访问元素，应使用vector或deque
 - 如果程序要求在容器的中间插入或者删除元素，应使用list或forward——list
 - 如果程序需要在头尾位置插入或者删除元素，但不会在中间位置进行插入或删除操作，则使用deque

- 如果程序只有在读取输入时才需要在容器中间位置插入元素，随后需要随机访问元素，则
 - 首先确认是否真的需要在容器中间位置插入元素，当处理输入数据时，通常可以很容易地向vector追加数据，然后再调用标准库的sort函数来重排容器中的元素，从而避免在中间位置添加元素。
 - 如果必须在中间位置插入元素，考虑在输入阶段使用list，一旦输入完成，将list中的内容拷贝到一个vector中
- 如果你不确定应该使用那种容器，那么可以在程序中只使用vector和list公共的操作，使用迭代器不使用下标操作，避免随机访问，这样在必要的时候使用vector或list都很方便

• 容器库概览

- 操作层次
 - 某些操作是所有容器类型都提供的
 - 另外一些操作只针对顺序容器，关联容器，无序容器
 - 还有一些操作只适用于一小部分容器
- 一般来说，每个容器都定义在一个头文件中，文件名与类型名相同。
- 对于大多数容器，但不是所有容器，我们还需要额外提供元素类型信息

```
list<Sales_data>      // 保存 Sales_data 对象的 list
deque<double>        // 保存 double 的 deque
```

- 顺序容器几乎可以保存任意类型的元素，特别是，我们可以定义一个容器，其元素的类型是另外一个容器
- 虽然我们可以在容器中保存几乎任何类型，但某些容易操作对元素类型有自己特殊的要求
- 操作列表

表 9.2: 容器操作	
类型别名	
iterator	此容器类型的迭代器类型
const_iterator	可以读取元素，但不能修改元素的迭代器类型
size_type	无符号整数类型，足够保存此种容器类型最大可能容器的大小
difference_type	带符号整数类型，足够保存两个迭代器之间的距离
value_type	元素类型
reference	元素的左值类型：与 value_type 含义相同
const_reference	元素的 const 左值类型（即，const value_type）
构造函数	
Cc;	默认构造函数，构造空容器（array，参见第 301 页）
Cc1(c2);	构造 c2 的拷贝 c1
Cc(b, e);	构造 c，将迭代器 b 和 e 指定的范围内的元素拷贝到 c（array 不支持）
Cc(a, b, c...);	列表初始化 c
赋值与 swap	
c1 = c2	将 c1 中的元素替换为 c2 中元素
c1 = {a, b, c...}	将 c1 中的元素替换为列表中元素（不适用于 array）
a.swap(b)	交换 a 和 b 的元素
swap(a, b)	与 a.swap(b) 等价
大小	
c.size()	c 中元素的数目（不支持 forward_list）
c.max_size()	c 可保存的最大元素数目
c.empty()	若 c 中存储了元素，返回 false，否则返回 true
添加/删除元素（不适用于 array）	
注：在不同容器中，这些操作的接口都不相同	
c.insert(args)	将 args 中的元素拷贝进 c
c.emplace(inits)	使用 inits 构造 c 中的一个元素
c.erase(args)	删除 args 指定的元素
c.clear()	删除 c 中的所有元素，返回 void
关系运算符	
==, !=	所有容器都支持相等（不等）运算符
<, <=, >, >=	关系运算符（无序关联容器不支持）
获取迭代器	
c.begin(), c.end()	返回指向 c 的首元素和尾元素之后位置的迭代器
c.cbegin(), c.cend()	返回 const_iterator

反向容器的额外成员（不支持 forward_list）	
reverse_iterator	按逆序寻址元素的迭代器
const_reverse_iterator	不能修改元素的逆序迭代器
c.rbegin(), c.rend()	返回指向 c 的尾元素和首元素之前位置的迭代器
c.crbegin(), c.crend()	返回 const_reverse_iterator

容器的定义与初始化

- 每个容器都指定了一个默认构造函数
- 操作

表 9.3: 容器定义和初始化	
C c;	默认构造函数。如果 C 是一个 array, 则 c 中元素按默认方式初始化; 否则 c 为空
C c1(c2)	c1 初始化为 c2 的拷贝。c1 和 c2 必须是相同类型 (即, 它们必须是相同的容器类型, 且保存的是相同的元素类型; 对于 array 类型, 两者还必须具有相同大小)
C c1=c2	
C c{a,b,c,...}	c 初始化为初始化列表中元素的拷贝。列表中元素的类型必须与 C 的元素类型相容。对于 array 类型, 列表中元素数目必须等于或小于 array 的大小, 任何遗漏的元素都进行值初始化 (参见 3.3.1 节, 第 88 页)
C c={a,b,c,...}	
C c(b,e)	c 初始化为迭代器 b 和 e 指定范围中的元素的拷贝。范围中元素的类型必须与 C 的元素类型相容 (array 不适用)
只有顺序容器 (不包括 array) 的构造函数才能接受大小参数	
C seq(n)	seq 包含 n 个元素, 这些元素进行了值初始化; 此构造函数是 explicit 的 (参见 7.5.4 节, 第 265 页)。(string 不适用)
C seq(n,t)	seq 包含 n 个初始化为值 t 的元素

- 将一个容器初始化为另外一个容器的拷贝
 - 方法
 - 可以直接拷贝整个容器
 - 或者 (array 除外) 拷贝由一个迭代器对指定元素的范围
 - 当一个容器初始化为另外一个容器的拷贝时, 两个容器的容器类型和元素类型都必须相同
 - 当传递迭代器参数来拷贝一个范围的时候, 就不要求容器类型是相同的了
- 与顺序容器大小相关的构造函数
 - 只有顺序容器的构造函数才接受大小参数, 关联容器不支持
- 标准库 array 具有固定大小
 - 构造
 - 为了使用 array 类型, 我们必须指定元素类型和大小¹

```
array<int, 42>           // 类型为: 保存 42 个 int 的数组
array<string, 10>        // 类型为: 保存 10 个 string 的数组
```
 - 值得注意的是, 虽然我们不能对内置数组类型进行拷贝或对象赋值操作, 但是 array 并无此限制

赋值与拷贝

- 赋值
 - 右边容器的元素全部拷贝来
 - 容器大小与右边容器的原大小相等
 - 与内置数组不同的是标准库 array 支持 array 之间赋值, 支持初始化, 但是不支持直接的数字赋值

```
array<int, 10> a1 = {0,1,2,3,4,5,6,7,8,9};
array<int, 10> a2 = {0}; // 所有元素值均为 0
a1 = a2; // 替换 a1 中的元素
a2 = {0}; // 错误：不能将一个花括号列表赋予数组
```

◇ array类型不支持assign

□ assign的使用（仅仅是顺序容器）

- ◆ 赋值运算符要求左边和右边的运算对象具有相同的类型，它将右边运算对象中所有元素拷贝到左边的运算对象中
- ◆ 顺序容器还等了一个名字为assign的成员，允许我们从一个不同但相容的类型赋值，或者从容器的一个子序列赋值，assign操作用参数所指定的元素（的拷贝）替换左边容器中所有的元素

◆ assign的两个版本

- ◇ 第一个版本可以传入两个迭代器，将元素指定为迭代器中指定元素的拷贝

```
list<string> names;
vector<const char*> oldstyle;
names = oldstyle; // 错误：容器类型不匹配
// 正确：可以将 const char*转换为 string
names.assign(oldstyle.cbegin(), oldstyle.cend());
```

- ◇ 第二个版本接受一个整型值和一个元素值。它用指定数目且具有相同给定值的元素替换容器中原有的元素

```
// 等价于 slist1.clear();
// 后跟 slist1.insert(slist1.begin(), 10, "Hiya!");
list<string> slist1(1); // 1个元素，为空 string
slist1.assign(10, "Hiya!"); // 10个元素，每个都是“Hiya!”
```

▪ 交换

□ 使用swap

- ◆ swap操作交换两个相同类型容器的内容。调用swap之后，两个容器中元素将会交换
- ◆ 调用swap时候，元素本身并未交换，swap只是交换了两个容器的内部的数据结构，因此除了array外，交换两个容器的内容保证会很快
 - ◇ 元素不会被移动意味着，除了string外，指向容器的迭代器，引用和指针在swap操作之后都不会失效，他们仍然指向swap操作之前所指向的哪些元素。但是在swap之后这些元素已经属于不同的容器了
- ◆ swap与array
 - ◇ 对于array来说，swap两个array会真正交换他们的元素，因此交换两个array所需的时间与array中元素的数目成正比
 - ◇ 因此对于array来说，在swap操作之后，指针，引用和迭代器所绑定的元素保持不变，，但元素值已经与另外一个array中对应的元素进行了交换
- ◆ 统一使用非成员版本的swap是一个好习惯

▪ 容器的赋值/交换运算

表 9.4: 容器赋值运算	
<code>c1=c2</code>	将 <code>c1</code> 中的元素替换为 <code>c2</code> 中元素的拷贝。 <code>c1</code> 和 <code>c2</code> 必须具有相同的类型
<code>c={a,b,c...}</code>	将 <code>c1</code> 中元素替换为初始化列表中元素的拷贝 (<code>array</code> 不适用)
<code>swap(c1,c2)</code> <code>c1.swap(c2)</code>	交换 <code>c1</code> 和 <code>c2</code> 中的元素。 <code>c1</code> 和 <code>c2</code> 必须具有相同的类型。 <code>swap</code> 通常比从 <code>c2</code> 向 <code>c1</code> 拷贝元素快得多
assign 操作不适用于关联容器和 <code>array</code>	
<code>seq.assign(b,e)</code>	将 <code>seq</code> 中的元素替换为迭代器 <code>b</code> 和 <code>e</code> 所表示的范围中的元素。迭代器 <code>b</code> 和 <code>e</code> 不能指向 <code>seq</code> 中的元素
<code>seq.assign(il)</code>	将 <code>seq</code> 中的元素替换为初始化列表 <code>il</code> 中的元素
<code>seq.assign(n,t)</code>	将 <code>seq</code> 中的元素替换为 <code>n</code> 个值为 <code>t</code> 的元素
	赋值相关运算会导致指向左边容器内部的迭代器、引用和指针失效。而 <code>swap</code> 操作将容器内容交换不会导致指向容器的迭代器、引用和指针失效 (容器类型为 <code>array</code> 和 <code>string</code> 的情况除外)。

- 容器的大小操作
 - `forward—list` 不支持 `size`，但是支持 `max—size` 和 `empty`，其他的都支持
- 关系运算符
 - 每个容器类型都支持相等运算符 (`==`, `!=`)
 - 除了无序关联容器外的所有容器都支持关系运算符 (`>`, `>=`, `<`, `<=`)
 - 关系运算符左右两边的运算对象必须是相同类型的容器，且必须保存相同类型的元素
 - 比较法则
 - 如果两个容器具有相同大小且所有元素都两两对应相等，则这两个容器相等，否则两个容器不等
 - 如果两个元素大小不同，但较小容器中每个元素都等于较大容器中对对应元素，则较小容器小于较大容器
 - 如果两个容器都不是另一个容器的前缀子序列，则它们的比较结果取决于第一个不相等的元素比较结果
 - 容器的关系运算符使用元素的关系运算符完成比较
 - 只有当其元素类型也定义了相应的比较运算符时，我们才可以使用关系运算符来比较容器
 - 比如如果自定义类型就无法比较大小。
- 顺序容器操作
 - 向顺序容器中添加元素
 - 操作
 - ◆ 除了 `array` 外，所有标准库容器都提供灵活的内存管理。在运行时候可以动态添加或删除元素来改变容器大小

表 9.5: 向顺序容器添加元素的操作	
这些操作会改变容器的大小; array 不支持这些操作。 forward_list 有自己专有版本的 insert 和 emplace; 参见 9.3.4 节 (第 312 页)。 forward_list 不支持 push_back 和 emplace_back。 vector 和 string 不支持 push_front 和 emplace_front。	
c.push_back(t) c.emplace_back(args)	在 c 的尾部创建一个值为 t 或由 args 创建的元素。返回 void
c.push_front(t) c.emplace_front(args)	在 c 的头部创建一个值为 t 或由 args 创建的元素。返回 void
c.insert(p,t) c.emplace(p,args)	在迭代器 p 指向的元素之前创建一个值为 t 或由 args 创建的元素。返回指向新添加的元素的迭代器
c.insert(p,n,t)	在迭代器 p 指向的元素之前插入 n 个值为 t 的元素。返回指向新添加的第一个元素的迭代器; 若 n 为 0, 则返回 p
c.insert(p,b,e)	将迭代器 b 和 e 指定的范围内的元素插入到迭代器 p 指向的元素之前。b 和 e 不能指向 c 中的元素。返回指向新添加的第一个元素的迭代器; 若范围为空, 则返回 p
c.insert(p,il)	il 是一个花括号包围的元素值列表。将这些给定值插入到迭代器 p 指向的元素之前。返回指向新添加的第一个元素的迭代器; 若列表为空, 则返回 p
 向一个 vector、string 或 deque 插入元素会使所有指向容器的迭代器、引用和指针失效。	

- pushback
 - ◆ 将一个元素追加到一个vector的尾部
 - ◆ 除了array和forward—list之外, 每个顺序容器都支持pushback
v.push—back (xxx)
- pushfront
 - ◆ list类容器, deque容器支持名为push—front的类似操作, 将元素插入到容器头部
 - ◆ 注意, deque像vector一样提供了随机访问元素的能力, 但它提供了vector所不支持的pushfront。
 - ◆ 在deque首尾之外的位置插入元素会很耗时
- insert
 - ◆ 虽然某些容器不支持push—front操作, 但是它们对于insert操作并无类似的限制
 - ◆ 因此我们可以利用insert将元素插入到容器的开始, 而不必担心容器是否支持pushfront
 - ◇ 但是这么做会很耗时
 - ◆ insert还可以插入更多的值。
- emplace
 - ◆ emplace front, emplace back, emplace这些操作分别对应pushfront, pushback, insert, 允许我们将元素放入容器头部, 容器尾部或者一个指定位置
 - ◆ 与push不同的是, push是对一个元素复制一下拷贝进去, emplace是直接利用参数构造一个新的元素, 将元素放入, 两者有本质上的区别
- 注意
 - ◆ 容器元素是拷贝
 - ◇ 当我们用一个对象来初始化容器时, 或将一个对象插入到容器中时, 实际上放入到容器中的对象值是一个拷贝, 而非对象本身

▪ 访问元素

- 包括array在内的每个顺序容器都有一个front成员函数，而除了forward—list之外所有顺序容器都有一个back成员函数，这两个操作分别返回首元素和为元素的引用
- 操作
 - ◆ 如果容器是一个const对象，则返回一个const引用

表 9.6: 在顺序容器中访问元素的操作	
at 和下标操作只适用于 string、vector、deque 和 array。 back 不适用于 forward_list。	
c.back()	返回 c 中尾元素的引用。若 c 为空，函数行为未定义
c.front()	返回 c 中首元素的引用。若 c 为空，函数行为未定义
c[n]	返回 c 中下标为 n 的元素的引用，n 是一个无符号整数。若 $n \geq c.size()$ ，则函数行为未定义
c.at(n)	返回下标为 n 的元素的引用。如果下标越界，则抛出一 out_of_range 异常
 对一个空容器调用 front 和 back，就像使用一个越界的下标一样，是一种严重的程序设计错误。	

- ◆ 与往常一样，如果我们使用auto变量来保存这些函数的返回值，并且希望使用此变量来改变元素的值，必须记得将变量定义为引用类型
- 下标操作的安全随机访问
 - ◆ 提供快速访问的随机容器也都提供下标运算符
 - ◆ 下标越界是一种编译器也查不出来的严重程序设计错误，可以使用at函数确保下标合法

▪ 删除元素

- 对于非array容器也有多种删除元素的方式
- 操作
 - ◆ 删除元素的成员函数并不检查其参数，在删除元素之前，程序员必须确保它们是存在的

表 9.7: 顺序容器的删除操作	
这些操作会改变容器的大小，所以不适用于 array。 forward_list 有特殊版本的 erase，参见 9.3.4 节（第 312 页）。 forward_list 不支持 pop_back；vector 和 string 不支持 pop_front。	
c.pop_back()	删除 c 中尾元素。若 c 为空，则函数行为未定义。函数返回 void
c.pop_front()	删除 c 中首元素。若 c 为空，则函数行为未定义。函数返回 void
c.erase(p)	删除迭代器 p 所指定的元素，返回一个指向被删元素之后元素的迭代器，若 p 指向尾元素，则返回尾后（off-the-end）迭代器。若 p 是尾后迭代器，则函数行为未定义
c.erase(b,e)	删除迭代器 b 和 e 所指定范围内的元素。返回一个指向最后一个被删元素之后元素的迭代器，若 e 本身就是尾后迭代器，则函数也返回尾后迭代器
c.clear()	删除 c 中的所有元素。返回 void
 删除 deque 中除首尾位置之外的任何元素都会使所有迭代器、引用和指针失效。指向 vector 或 string 中删除点之后位置的迭代器、引用和指针都会失效。	

- popfront和popback
 - ◆ popfront和popback与push类型一样不支持vector和string
 - ◆ 不能对一个空的容器执行弹出操作
 - ◆ 这些操作返回void，如果你需要弹出元素的值，就必须在执行弹出操作之前保存他

□ 从容器内部删除一个元素

- ◆ 我们可以使用erase删除一个或者多个元素（forward—list除外）
(erase为前闭后开式删除)

如果两个迭代器 elem1 和 elem2 相等，则什么也不会发生，容器保持不变。
哪怕两个迭代器是指向尾后位置（例如 end()+1），也是如此，程序也不会出错。

因此 elem1 和 elem2 都是尾后迭代器时，容器保持不变。

如果 elem2 为尾后迭代器，elem1 指向之前的合法位置，则会删除从 elem1 开始直至容器末尾的所有元素。

- ◆ 特殊的forwardlist操作

◇ 操作

- ▶ beforebegin：这个迭代器允许我们在链表首元素之前并不存在的元素之后添加或删除元素

表 9.8: 在 forward_list 中插入或删除元素的操作

lst.before_begin()	返回指向链表首元素之前不存在的元素的迭代器。此迭代器不能解引用。cbefore_begin() 返回一个 const_iterator
lst.insert_after(p, t)	在迭代器 p 之后的位置插入元素。t 是一个对象，n 是数量，b 和 e 是表示范围的一对迭代器（b 和 e 不能指向 lst 内），il 是一个花括号列表。返回一个指向最后一个插入元素的迭代器。如果范围为空，则返回 p。若 p 为尾后迭代器，则函数行为未定义
lst.insert_after(p, n, t)	
lst.insert_after(p, b, e)	
lst.insert_after(p, il)	
emplace_after(p, args)	使用 args 在 p 指定的位置之后创建一个元素。返回一个指向这个新元素的迭代器。若 p 为尾后迭代器，则函数行为未定义
lst.erase_after(p)	删除 p 指向的位置之后的元素，或删除从 b 之后直到（但不包含）e 之间的元素。返回一个指向被删元素之后元素的迭代器，若不存在这样的元素，则返回尾后迭代器。如果 p 指向 lst 的尾元素或者是一个尾后迭代器，则函数行为未定义
lst.erase_after(b, e)	

▪ 改变容器大小

□ 操作

- ◆ 我们使用resize来改变容器的大小（array不支持）

表 9.9: 顺序容器大小操作

resize 不适用于 array	
c.resize(n)	调整 c 的大小为 n 个元素。若 n<c.size(), 则多出的元素被丢弃。若必须添加新元素，对新元素进行值初始化
c.resize(n, t)	调整 c 的大小为 n 个元素。任何新添加的元素都初始化为值 t
	如果 resize 缩小容器，则指向被删除元素的迭代器、引用和指针都会失效；对 vector、string 或 deque 进行 resize 可能导致迭代器、指针和引用失效。

- ◆ vector 的reserve增加了vector的capacity，但是它的size没有改变！
而resize改变了vector的capacity同时也增加了它的size！
- ◆ reserve是容器预留空间，但在空间内不真正创建元素对象，所以在没有添加新的对象之前，不能引用容器内的元素。加入新的元素时，要调用push_back()/insert()函数。

□ 注意

对于元素是类类型，则单参数 resize 版本要求该类型必须提供一个默认构造函数。

▪ 容器操作可能使迭代器失效

□ 添加

- ◆ 如果容器是vector或string，且存储空间被重新分配，则指向容器的迭代器、指针和引用都会失效。如果存储空间未重新分配，指向插入位置之前的元素的迭代器、指针和引用仍有效，但指向插入位置之后元素的

迭代器、指针和引用将会失效。

- ◆ 对于deque，插入到除首尾位置之外的任何位置都会导致迭代器、指针和引用失效。如果在首尾位置添加元素，迭代器会失效，但指向存在的元素的引用和指针不会失效。
- ◆ 对于list和forward_list，指向容器的迭代器（包括尾后迭代器和首前迭代器）、指针和引用仍有效。

□ 删除

- ◆ 对于list和forward_list，指向容器其他位置的迭代器（包括尾后迭代器和首前迭代器）、引用和指针仍有效。
- ◆ 对于deque，如果在首尾之外的任何位置删除元素，那么指向被删除元素外其他元素的迭代器、引用或指针也会失效。如果是删除deque的尾元素，则尾后迭代器也会失效，但其他迭代器、引用和指针不受影响；如果是删除首元素，这些也不会受影响。
- ◆ 对于vector和string，指向被删元素之前元素的迭代器、引用和指针仍有效。注意：当我们删除元素时，尾后迭代器总是会失效。

□ 注意

- ◆ 使用失效的迭代器、指针或引用是严重的运行时错误。
- ◆ 当你使用迭代器（或指向容器元素的引用或指针）时，最小化要求迭代器必须保持有效的程序片段是一个好的方法。
- ◆ 由于向迭代器添加元素和从迭代器删除元素的代码可能会使迭代器失效，因此必须保证每次改变容器的操作之后都正确地重新定位迭代器。这个建议对vector、string和deque尤为重要。

■ 编写改变容器的循环的程序

- 在调用erase后，不必递增迭代器，因为erase返回的迭代器已经指向序列中下一个元素
- 调用insert后，需要递增迭代器两次

■ 管理容量的成员函数

□ 操作

表 9.10：容器大小管理操作	
shrink_to_fit 只适用于 vector、string 和 deque。	
capacity 和 reserve 只适用于 vector 和 string。	
◆ c.shrink_to_fit()	请将 capacity() 减少为与 size() 相同大小
c.capacity()	不重新分配内存空间的话，c 可以保存多少元素
c.reserve(n)	分配至少能容纳 n 个元素的内存空间

- reverse并不改变容器中元素的数量，它仅仅影响vector预先分配多大的内存空间
- 只有当需要的内存空间超过当前容量时，reverse调用才会改变vector的容量，小于等于不会对reverse的容量有任何的改变
 - ◆ 我们可以使用shrinktofit来要求deque，vector，string退回不需要的空间，具体的实现可以忽略此请求
- capacity操作告诉我们容器在不扩容的情况下可以容纳多少个元素，reserve操作允许我们通知容器他应该准备保存多少个元素

□ 注意

- ◆ 容器的size是指他现在有多少元素，但是capacity是指在不分配新的内存空间的前提下最多可以保存多少元素

▪ 额外的string操作

□ 构造

◆ 操作

◇ 主要是利用下标进行的一些操作

表 9.11: 构造 string 的其他方法	
n、len2 和 pos2 都是无符号值	
string s(cp,n)	s 是 cp 指向的数组中前 n 个字符的拷贝。此数组至少应该包含 n 个字符
string s(s2,pos2)	s 是 string s2 从下标 pos2 开始的字符的拷贝。若 pos2>s2.size(), 构造函数的行为未定义
string s(s2,pos2,len2)	s 是 string s2 从下标 pos2 开始 len2 个字符的拷贝。若 pos2>s2.size(), 构造函数的行为未定义。不管 len2 的值是多少, 构造函数至多拷贝 s2.size()-pos2 个字符

◇ substr操作：利用下标拷贝一部分string出来，他是原始string的一部分或全部的拷贝

表 9.12: 子字符串操作	
s.substr(pos,n)	返回一个 string, 包含 s 中从 pos 开始的 n 个字符的拷贝。pos 的默认值为 0。n 的默认值为 s.size()-pos, 即拷贝从 pos 开始的所有字符

▪ 注意

- 无论是添加还是删除元素，都会使原有的迭代器失效
- 不要保存end返回的迭代器
- 如果在一个循环中插入/删除deque/string或vector中的元素，不要缓存end返回的迭代器

○ string搜索操作

▪ 操作

- 如果搜索到了，返回一个unsigned类型下标，搜索失败返回一个string::npos，默认为-1，所以不要用int类型去保存搜索返回的结果

表 9.14: string 搜索操作	
搜索操作返回指定字符出现的下标，如果未找到则返回 npos。	
s.find(args)	查找 s 中 args 第一次出现的位置
s.rfind(args)	查找 s 中 args 最后一次出现的位置
s.find_first_of(args)	在 s 中查找 args 中任何一个字符第一次出现的位置。
s.find_last_of(args)	在 s 中查找 args 中任何一个字符最后一次出现的位置
s.find_first_not_of(args)	在 s 中查找第一个不在 args 中的字符
s.find_last_not_of(args)	在 s 中查找最后一个不在 args 中的字符

args 必须是以下形式之一	
c,pos	从 s 中位置 pos 开始查找字符 c。pos 默认为 0
s2,pos	从 s 中位置 pos 开始查找字符串 s2。pos 默认为 0
cp,pos	从 s 中位置 pos 开始查找指针 cp 指向的以空字符结尾的 C 风格字符串。pos 默认为 0
cp,pos,n	从 s 中位置 pos 开始查找指针 cp 指向的数组的前 n 个字符。pos 和 n 无默认值

- 搜索操作是对大小写敏感的
- find查找字符串，findxx查找具体的信息

○ string的比较，数值转换函数

▪ 比较函数

- C++ 中的比较函数可以比较不止整个字符串，还可以对部分字符串进行比较

表 9.15: s.compare 的几种参数形式	
s2	比较 s 和 s2
pos1, n1, s2	将 s 中从 pos1 开始的 n1 个字符与 s2 进行比较
pos1, n1, s2, pos2, n2	将 s 中从 pos1 开始的 n1 个字符与 s2 中从 pos2 开始的 n2 个字符进行比较
cp	比较 s 与 cp 指向的以空字符结尾的字符数组
pos1, n1, cp	将 s 中从 pos1 开始的 n1 个字符与 cp 指向的以空字符结尾的字符数组进行比较
pos1, n1, cp, n2	将 s 中从 pos1 开始的 n1 个字符与指针 cp 指向的地址开始的 n2 个字符进行比较

▪ 转换函数

- 如果 string 不能转换为一个数值，这些函数抛出一个 invalid_argument 异常，如果转换得到的数值无法用任何类型来表示，则抛出一个 out of range 的异常

表 9.16: string 和数值之间的转换	
to_string(val)	一组重载函数，返回数值 val 的 string 表示。val 可以是任何算术类型（参见 2.1.1 节，第 30 页）。对每个浮点类型和 int 或更大的整型，都有相应版本的 to_string。与往常一样，小整型会被提升（参见 4.11.1 节，第 142 页）
stoi(s, p, b)	返回 s 的起始子串（表示整数内容）的数值，返回值类型分别是 int、long、unsigned long、long long、unsigned long long。b 表示转换所用的基数，默认值为 10。p 是 size_t 指针，用来保存 s 中第一个非数值字符的下标，p 默认为 0，即，函数不保存下标
stol(s, p, b)	
stoul(s, p, b)	
stoll(s, p, b)	
stoull(s, p, b)	返回 s 的起始子串（表示浮点数内容）的数值，返回值类型分别是 float、double 或 long double。参数 p 的作用与整数转换函数中一样
stof(s, p)	
stod(s, p)	
stold(s, p)	

○ 栈适配器

▪ 操作

- 利用容器适配器的操作，我们可以定义栈（最初是用 deque 实现的）

表 9.18: 表 9.17 未列出的栈操作	
栈默认基于 deque 实现，也可以在 list 或 vector 之上实现。	
s.pop()	删除栈顶元素，但不返回该元素值
s.push(item)	创建一个新元素压入栈顶，该元素通过拷贝或移动 item 而来，或者由 args 构造
s.emplace(args)	由 args 构造
s.top()	返回栈顶元素，但不将元素弹出栈

○ 队列适配器

▪ 操作

- 队列采用先进先出的储存和访问策略

表 9.19: 表 9.17 未列出的 queue 和 priority_queue 操作	
queue 默认基于 deque 实现，priority_queue 默认基于 vector 实现；queue 也可以用 list 或 vector 实现，priority_queue 也可以用 deque 实现。	
q.pop()	返回 queue 的首元素或 priority_queue 的最高优先级的元素，但不删除此元素
q.front()	返回首元素或尾元素，但不删除此元素
q.back()	只适用于 queue

q.top()	返回最高优先级元素，但不删除该元素 只适用于 priority_queue
q.push(item)	在 queue 末尾或 priority_queue 中恰当的位置创建一个元素，其值为 item，或者由 args 构造
q.emplace(args)	

• 迭代器

- 与容器一样，迭代器有着公共的接口：如果一个迭代器提供某个操作，那么所有提供相同操作的迭代器对这个操作的实现方式都是相同的。
- forward—list不支持递减运算符
- 迭代器范围
 - 一个迭代器范围由一对迭代器表示，两个迭代器分别指向同一个容器中元素或者是尾元素之后的位置。
 - 范围被称为左闭合区间
【begin, end)】
- begin和end成员
 - begin和end有多个版本，带r的反向迭代器和带c的const迭代器
 - begin是被重载过的，一个可以返回const迭代器，一个可以返回普通迭代器
 - cbegin, end返回const类型，begin和end返回普通的iter类型

• 容器适配器

- 适配器是标准库中一个通用的概念，容器，迭代器都有适配器，本质上，一个适配器是一种机制，能使某种事物看起来像另外一种事物一样
- 操作

表 9.17：所有容器适配器都支持的操作和类型	
size_type	一种类型，足以保存当前类型的最大对象的大小
value_type	元素类型
container_type	实现适配器的底层容器类型
A a;	创建一个名为 a 的空适配器
A a(c);	创建一个名为 a 的适配器，带有容器 c 的一个拷贝
关系运算符	每个适配器都支持所有关系运算符：==、!=、<、<=、>和>= 这些运算符返回底层容器的比较结果
a.empty()	若 a 包含任何元素，返回 false，否则返回 true
a.size()	返回 a 中的元素数目
swap(a,b)	交换 a 和 b 的内容，a 和 b 必须有相同类型，包括底层容器类型也必须相同
a.swap(b)	

- 定义一个适配器
 - 每个适配器都定义了两个构造函数，默认构造函数创建一个空对象，接受一个容器的构造函数拷贝该容器来初始化适配器
stack<int> stk(deq)
 - 在不明确指定重载类型的对象的时候，我们默认基于deque构造，我们可以直接添加一个函数进行指定构造
stack<string,vector<string>> a...

- 当我们使用添加和删除元素的容器操作时，必须注意这些操作可能使指向容器中元素的迭代器，指针或引用失效，很多会使迭代器失效的操作，如insert和erase，都会返回一个新的迭代器，来帮助程序员维护容器中的位置。如果循环程序中改变了容器的大小的操作就要尤其小心其中迭代器，指针和引用的使用